Pruebas de rendimiento de ASTER (E1–E6)

Proyecto: ASTER — Asistente Virtual Multimodal de Escritorio
Versión del informe: 1.0
Fecha: 8 de noviembre de 2026
Ámbito: evaluación de rendimiento del frontend (React + TypeScript), del backend nativo (Rust + Tauri 2) y del uso real de la aplicación.

---

1. Resumen ejecutivo

Se implementaron y ejecutaron seis pruebas de rendimiento sobre el código de ASTER. Las pruebas E1–E4 son microbenchmarks deterministas (sin red ni claves de IA); E5 analiza las métricas reales que la aplicación ya registra; E6 mide el arranque y el consumo de recursos del ejecutable de release.

Prueba	Qué mide	Resultado principal	Veredicto
E1 Enrutado local	`fastRoute`, `extractFileSearch`, `parseRoutedIntent`	1,1–2,6 µs por orden; lote de 7 en 14,0 µs	**Cumple** (umbral 1 ms)
E2 Búsqueda de archivos	`search_files` en Rust sobre árboles de 1k, 5k y 20k entradas	277,3 ms (1k), 1,13 s (5k) y **4,47 s (20k)**	**No cumple** (umbral 300 ms)
E3 Parsers de búsqueda web	`parse_duckduckgo`, `parse_bing`, `html_to_text` sobre 500 KB	76,1 µs, 110,0 µs y 5,48 ms	**Cumple**
E4 Orquestación	`AsterController.sendText` con IA e IPC simulados	5,7–8,7 µs por orden (p95 ≤ 28,9 µs)	**Cumple** (umbral 100 µs)
E5 Latencia extremo a extremo	58 interacciones reales y 42 entradas de auditoría	p50 de 0,7–2,3 s; cola de hasta 16,4 s en archivos	**Parcial**
E6 Arranque y recursos	Ejecutable de release, RAM, CPU en reposo	Arranque 1 190 ms; RAM 398 MB; CPU reposo 4,98 %	**Cumple** (umbral ≤ 5 s / 500 MB / 5 %)

Hallazgos principales

1. **El enrutado local y la orquestación no son el cuello de botella.** Resolver una orden obvia cuesta microsegundos; el tiempo percibido lo domina la red, el disco o el lanzamiento de aplicaciones.
2. **La búsqueda de archivos es el problema real.** `search_files` tarda 4,47 s en un árbol de 20 000 entradas porque pide `symlink_metadata` de **cada** entrada antes de comparar el nombre. Esa llamada cuesta ~127 µs por entrada en este equipo, frente a 0,6–1,2 µs de `entry.metadata()` o `entry.file_type()`.

3. **Los parsers web están sanos.** Procesan una página de 500 KB en menos de 0,11 ms (DuckDuckGo/Bing) y convierten el HTML completo a texto en 5,5 ms.
4. **La fiabilidad de los proveedores pesa más que su latencia.** El 33 % de éxito de `web_search` se explica por cuotas 429 y modelos retirados, no por lentitud del código.
5. **El bundle principal roza el límite de Vite.** 801,3 KiB de JavaScript en un solo fragmento (820,5 kB según Vite; 209,8 kB gzip); Vite avisa de *chunks* por encima de 500 kB.

---

2. Objetivos e indicadores

Objetivo general

Cuantificar el rendimiento de ASTER en sus capas críticas, detectar cuellos de botella con evidencia reproducible y dejar un procedimiento que pueda repetirse en otro equipo.

Objetivos específicos e indicadores

Indicador	Prueba	Criterio de aceptación	Medido
PI1 Enrutado local	E1	Mediana ≤ 1 ms por orden y lote de 7 ≤ 10 ms	1,1–2,6 µs; lote 14,0 µs
PI2 Búsqueda de archivos	E2	Árbol de 20 000 entradas ≤ 300 ms	4,47 s
PI3 Parseo web	E3	Parsers ≤ 1 ms y HTML→texto ≤ 10 ms por 500 KB	0,076–0,110 ms; 5,48 ms
PI4 Orquestación	E4	≤ 100 µs por orden	5,7–8,7 µs
PI5 Latencia percibida	E5	p95 de interacciones exitosas ≤ 5 s	1,28–16,4 s según acción
PI6 Arranque y recursos	E6	Arranque ≤ 5 s; RAM ≤ 500 MB; CPU reposo ≤ 5 %	Arranque 1 190 ms ✓; RAM 398 MB ✓; CPU 4,98 % ✓

El umbral de 300 ms de PI2 se fijó antes de ejecutar la prueba; no se ajustó después de conocer el resultado. Los demás umbrales son criterios de aceptación razonables para un asistente de escritorio y se declaran aquí para que el lector pueda juzgar el veredicto.

---

3. Metodología

3.1 Dos niveles de medición

- **Microbenchmarks (E1–E4):** miden una función o un flujo concreto, aíslan la red y el IPC, y permiten comparar cambios de código con poco ruido.
- **Observación de uso real (E5):** aprovecha `metrics.jsonl` y `audit.jsonl`, que la aplicación ya escribe. Refleja el comportamiento percibido, pero no es un experimento controlado.
- **Recursos del ejecutable (E6):** mide el artefacto que recibe el usuario, no el servidor de desarrollo.

3.2 IA simulada como corrida principal

No se ejecutaron llamadas reales a proveedores de IA para E1–E4:

- E1 y E3 son funciones puras o de Rust sin red.
- E4 sustituye `interpret`, `chat`, `webSearch`, el IPC de Tauri, la auditoría y el historial por dobles de prueba. Así se mide **solo** la sobrecarga del controlador, no la latencia del proveedor.
- E5 usa datos ya existentes; no genera consumo nuevo.

El guion para una corrida con IA real queda en `scripts/perf/scenarios.json`: 10 órdenes fijas × 3 rondas, con calentamiento y análisis mediante `npm run perf:report`. Se ejecutará cuando haya presupuesto de cuota; su ausencia no invalida E1–E4.

3.3 Equipo y software

Dato	Valor
Sistema	Microsoft Windows 11 Pro 10.0.26200
CPU	Intel Core i7-1165G7 @ 2,80 GHz (4 núcleos / 8 hilos)
RAM	15,75 GB
Node.js / npm	v24.14.1 / 11.5.2
Rust	1.98.0
Frontend	React 19.1, TypeScript 5.8, Vite 7.3.6
Pruebas	Vitest 5.0.3, tinybench 6.2.0, criterion 0.5.1
Escritorio	Tauri 2.12

3.4 Higiene de medición

- Los árboles sintéticos de E2 se crean en `%TEMP%`, fuera de OneDrive, para que la sincronización del escritorio no contamine la medición.
- E2 y E3 usan el perfil `perf` de Cargo (`opt-level = 3`, sin LTO, `panic = unwind`), porque el perfil `release` usa `panic = "abort"` y LTO.
- E1 y E4 se ejecutaron con un solo hilo de Vitest (`fileParallelism: false`).

- Se reportan medianas y p95; los máximos se citan solo cuando explican una cola.
- Las corridas finales se hicieron sin compilaciones ni benchmarks en paralelo.

3.5 Reproducción

```
npm test                # 9 archivos / 117 pruebas
npm run build           # typecheck + bundle
cargo test              # 22 pruebas + 1 ignorada (red)
cargo build             # backend de desarrollo

npm run bench           # E1 y E4 (tinybench)
npm run bench:rust      # E2 y E3 (criterion, perfil perf)
npm run perf:criterion  # tablas de criterion
npm run perf:report     # E5
npm run perf:recursos   # E6
```

Los resultados crudos quedan en `docs/perf/`.

---

4. E1 — Enrutado local (cero tokens)

4.1 Qué se mide

`fastRoute` decide localmente las órdenes obvias (abrir una app, buscar un archivo, una carpeta, una búsqueda web, un saludo, el historial o el sistema) sin consultar al modelo. `extractFileSearch` extrae consulta, ámbito y `foldersOnly`. `parseRoutedIntent` traduce el JSON del router LLM.

Una degradación aquí afectaría a **todas** las órdenes, incluso a las que no usan IA.

4.2 Resultados

Caso	operaciones/s	Mediana	p95
`fastRoute` · abrir app	821 327	1,1 µs	2,3 µs
`fastRoute` · buscar archivo	714 593	1,2 µs	5,4 µs
`fastRoute` · buscar carpeta	515 352	2,1 µs	4,7 µs
`fastRoute` · búsqueda web	461 762	2,6 µs	5,3 µs
`fastRoute` · charla	518 320	2,5 µs	4,9 µs
`fastRoute` · historial	610 412	2,0 µs	4,4 µs
`fastRoute` · sistema	474 363	2,4 µs	5,3 µs
`fastRoute` · **lote de 7 órdenes**	87 295	**14,0 µs**	33,9 µs
`extractFileSearch` · 5 consultas	687 757–999 701	1,2–1,5 µs	≤ 3,2 µs
`parseRoutedIntent` · `find_files` válido	763 501	1,4 µs	3,3 µs
`parseRoutedIntent` · conversación	1 147 250	1,1 µs	2,5 µs
`parseRoutedIntent` · JSON roto	33 475	36,4 µs	78,6 µs

Muestras: entre 12 982 y 377 225 iteraciones por tarea (tinybench, 500 ms por tarea más 150 ms de calentamiento).

4.3 Lectura

- El caso normal está **tres órdenes de magnitud** por debajo del umbral de 1 ms.
- El peor caso (`JSON roto`) tarda 36,4 µs porque recorre el texto y cae al respaldo; sigue siendo despreciable frente a una llamada de red.
- La vía rápida no es solo una optimización de cuota: también elimina la latencia de red de las órdenes más frecuentes.

Veredicto E1: cumple. El enrutado local no es un cuello de botella.

---

5. E2 — Búsqueda local de archivos

5.1 Diseño

`search_files` recorre las raíces autorizadas en anchura (BFS), poda carpetas de trabajo (`node_modules` , `.git` , `target` , etc.) y se detiene al llegar a `MAX_ENTRIES = 20 000` o al límite de resultados.

Se construyeron tres árboles reproducibles:

Árbol	Contenido	Entradas visitadas
1k	40 carpetas, 1 000 documentos, `pago.pdf`, `Andi/`, `Andi.zip`, cadena de 5 niveles	1 001
5k + poda	Como el anterior con 5 000 documentos y un `node_modules` de 2 000 archivos	7 001
20k	20 000 documentos y un objetivo al final de una cadena de 5 niveles	20 001

Consultas: `qqqq-inventado.xyz` (barrido completo sin coincidencias), `zz-objetivo.pdf` (acierto profundo) y `grupo-39` con `folders_only = true`. Antes de medir, el benchmark valida que el barrido devuelve 0 resultados y que el objetivo profundo y `grupo-39` se encuentran.

5.2 Resultados

Escenario	Mediana	IC 95 %	Coste por entrada
Barrido completo · 1k	277,31 ms	250,90–290,71 ms	277,0 µs
Barrido completo · 5k + poda	1 125,13 ms	1 106,10–1 177,89 ms	160,7 µs
Barrido completo · 20k	**4 465,69 ms**	4 432,35–4 657,83 ms	223,3 µs
Acierto profundo · 20k	4 675,29 ms	4 543,61–4 755,17 ms	233,8 µs
Solo carpetas · 5k + poda	**65,97 ms**	62,62–69,34 ms	**9,4 µs**

Muestras: 20 por escenario (criterion).

5.3 Diagnóstico del cuello de botella

El contraste entre «solo carpetas» (9,4 µs/entrada) y el barrido completo (161–277 µs/entrada) apunta a una llamada que solo se hace para archivos. En `files.rs`, `to_hit()` ejecuta `std::fs::symlink_metadata(path)` **antes** de comparar el nombre, y se invoca para todas las entradas.

Para aislar el coste se compiló una sonda en `%TEMP%` con 2 000 archivos:

Operación (2 000 entradas)	Tiempo	Coste por entrada
`read_dir` + `entry.file_type()`	2,32 ms	1,16 µs
`read_dir` + `entry.metadata()`	1,27 ms	0,64 µs
`read_dir` + `fs::symlink_metadata(entry.path())`	**253,82 ms**	**126,9 µs**

La diferencia es de dos órdenes de magnitud. En Windows, `symlink_metadata` abre cada archivo con `FILE_FLAG_OPEN_REPARSE_POINT`; `entry.metadata()` y `entry.file_type()` aprovechan la información que `read_dir` ya obtuvo. El antivirus y el filtro del sistema de archivos amplifican ese coste por apertura.

La prueba «solo carpetas» confirma la causa de forma indirecta: al saltarse los archivos antes de `to_hit`, baja de 1,13 s a 66 ms (17 veces menos coste por entrada) sin cambiar el recorrido.

5.4 Veredicto y recomendación

Veredicto E2: no cumple. El umbral era 300 ms para 20k y se midieron 4,47 s (14,9 veces por encima). No es un fallo del benchmark: la misma lentitud aparece en producción (E5, p95 de `find_files` = 16,4 s).

Mejoras recomendadas, en orden:

1. **Comparar el nombre antes de pedir metadatos.** Usar `entry.file_name()` y llamar a `to_hit` solo cuando haya coincidencia. El caso «solo carpetas» ya demuestra el orden de magnitud de la ganancia.
2. **Reutilizar los metadatos de `DirEntry`.** Pasar `entry.metadata()` o `entry.file_type()` a `to_hit` en vez de volver a abrir la ruta.
3. **Acotar el trabajo por tiempo o por raíz.** Un presupuesto de 1–2 s con resultados parciales evita que una carpeta enorme bloquee la interfaz.
4. **Cachear resultados por consulta y raíz** durante la sesión.

No se modificó la lógica de producción en esta entrega: el objetivo era medir y diagnosticar, no alterar el comportamiento del catálogo sin una prueba de regresión específica.

---

6. E3 — Parsers de búsqueda web

6.1 Diseño

`websearch.rs` descarga HTML de DuckDuckGo (o Bing como respaldo), recorta a 500 000 caracteres y extrae título, URL y resumen; después convierte la página en texto para resumirla con el modelo de chat.

Los fixtures sintéticos tienen ~500 KB, relleno primero y **6 resultados al final**, para forzar el recorrido completo del parser. Los benchmarks verifican que los 6 resultados se extraen antes de medir.

6.2 Resultados

Benchmark	Mediana	IC 95 %	Rendimiento
`parse_duckduckgo` · 500 KB	76,13 µs	72,13–84,02 µs	6 559,8 MB/s
`parse_bing` · 500 KB	109,96 µs	105,09–115,38 µs	4 541,4 MB/s
`html_to_text` · 500 KB	5,48 ms	5,21–5,73 ms	91,2 MB/s

Muestras: 50 por benchmark.

6.3 Lectura

- Extraer los resultados de una página es prácticamente gratis frente a la descarga de red (cientos de milisegundos).
- `html_to_text` recorre y reconstruye medio megabyte de HTML; 5,5 ms es un coste razonable y solo se paga una vez por página.
- El parser es lineal y no muestra un crecimiento desproporcionado.

Veredicto E3: cumple. El parseo no explica la latencia de `web_search`.

---

7. E4 — Orquestación del controlador

7.1 Diseño

`AsterController.sendText` es el único orquestador: interpreta, ejecuta en el catálogo, habla y mide. Para aislarlo se sustituyeron por dobles:

- `interpret`, `chat`, `webSearch`, `describeScreen`, `generateImage`;
- los comandos IPC (`openAuthorizedApp`, `findFiles`, etc.);
- auditoría, métricas e historial;
- el puerto de voz.

Cada iteración crea un controlador nuevo para no medir el crecimiento del historial.

7.2 Resultados

Orden	operaciones/s	Mediana	p95
Abrir app autorizada	109 932	8,0 µs	22,3 µs
Buscar archivos	101 472	8,7 µs	19,3 µs
Búsqueda web ya sintetizada	147 998	6,4 µs	12,7 µs
Conversación	148 309	5,7 µs	28,9 µs

Muestras: 39 932–66 017 iteraciones por tarea.

7.3 Lectura

- La sobrecarga propia (crear mensajes, cambiar estado, formatear, auditar y medir) es de **microsegundos**.
- Frente a una respuesta de IA de 1 s, el controlador representa menos del 1 %.
- El p95 de conversación (28,9 µs) refleja el coste de crear el controlador y encadenar promesas, no una fuga de estado.

Veredicto E4: cumple. Optimizar el controlador no movería la aguja; el trabajo debe ir a la red, al disco o al lanzamiento de aplicaciones.

---

8. E5 — Latencia extremo a extremo (observacional)

8.1 Naturaleza de los datos

Se analizaron **58 interacciones** de `metrics.jsonl` y **42 entradas** de `audit.jsonl` del equipo de desarrollo. No son una corrida controlada: son el uso normal de la app entre octubre y noviembre de 2026, con distintas versiones y configuraciones. Por eso se reportan como evidencia observacional.

El análisis separa:

- **todas** las interacciones, incluidas las fallidas;
- **solo éxito**, que es la serie que describe el rendimiento normal.

Una respuesta 429 de Gemini puede tardar 60 s en agotar reintentos y desplaza la media de toda la tabla; mezclarla con las exitosas daría una imagen falsa.

8.2 Totales por acción

Acción	n	Éxito	p50 éxito	p95 éxito	p50 todas	p95 todas
`find_files`	19	100 %	1 453 ms	16 384 ms	1 453 ms	16 384 ms
`web_search`	12	33,3 %	3 913 ms	5 706 ms	1 216 ms	60 459 ms
`conversation`	10	90 %	748 ms	1 279 ms	722 ms	1 279 ms
`open_authorized_app`	8	100 %	2 341 ms	8 319 ms	2 341 ms	8 319 ms
`system_info`	2	100 %	1 716 ms	3 556 ms	1 716 ms	3 556 ms
`create_file`	1	100 %	21 ms	—	21 ms	21 ms
`move_files`	1	100 %	21 ms	—	21 ms	21 ms
`automation:spotify`	1	100 %	309 ms	—	309 ms	309 ms
`create_folder`	2	0 %	—	—	15 ms	35 ms
`automation:open_url`	1	0 %	—	—	668 ms	668 ms
`cancelled_task`	1	0 %	—	—	0 ms	0 ms

8.3 Desglose por etapa

`interpretMs` es el enrutado. `executeMs` se mide desde el inicio de la orden, así que **incluye** la interpretación; `totalMs` añade STT y TTS.

Acción	interpret p50	interpret p95	execute p50	execute p95	TTS p50
`find_files`	45 ms	965 ms	1 452 ms	16 368 ms	1 ms
`web_search`	94 ms	60 154 ms	1 216 ms	60 459 ms	0 ms
`conversation`	2 ms	16 ms	718 ms	1 279 ms	1 ms
`open_authorized_app`	72 ms	8 313 ms	2 339 ms	8 318 ms	1 ms
`system_info`	28 ms	2 138 ms	1 688 ms	3 533 ms	17 ms

8.4 Auditoría y fallos

Acción	n	Estados	p50	p95	Máximo
`find_files`	19	success: 19	1 451 ms	16 367 ms	16 367 ms
`web_search`	10	error: 6, success: 4	1 148 ms	5 693 ms	5 693 ms
`open_authorized_app`	5	success: 5	2 038 ms	2 872 ms	2 872 ms
`create_folder`	2	error: 1, cancelled: 1	32 ms	9 739 ms	9 739 ms
`conversation`	1	error: 1	2 ms	2 ms	2 ms
`create_file`	1	success: 1	16 299 ms	16 299 ms	16 299 ms
`move_files`	1	success: 1	7 126 ms	7 126 ms	7 126 ms
`system_info`	1	success: 1	3 532 ms	3 532 ms	3 532 ms

De las 13 interacciones fallidas:

- **6 son 429** de cuota (Gemini).
- 1 es un **404** por un identificador de modelo retirado.
- 1 es `contents are required` (validación de la API).
- 1 es «ningún proveedor pudo atender» (cuota agotada en la cadena).
- 1 es una carpeta que ya existe.
- 1 es una URL con un esquema no permitido.
- 2 son cancelaciones explícitas.

Ninguno de esos fallos es un problema de rendimiento del código; son disponibilidad, configuración o validación.

La auditoría de `create_file` marca 16,3 s porque su reloj incluye la espera del diálogo de confirmación humana; la métrica de la interacción (21 ms) mide solo la preparación. No son comparables.

8.5 Veredicto

Veredicto E5: parcial.

- Las conversaciones (p50 748 ms, p95 1,28 s) cumplen de sobra el umbral de 5 s.
- Las acciones nativas típicas (p50 1,5–2,3 s) son aceptables.
- La **cola** incumple: `find_files` llega a 16,4 s y `open_authorized_app` a 8,3 s. La primera coincide con el cuello de botella de E2; la segunda es el coste de resolver y lanzar aplicaciones del sistema.
- `web_search` no falla por lentitud del código: cuando funciona, su p95 es 5,7 s, apenas por encima del umbral. Su problema es la tasa de éxito del 33 %.

El detalle por interacción, sin el texto dictado, está en `perf/raw/metrics-anon.jsonl`. El protocolo para repetir la medición con IA real está en `scripts/perf/scenarios.json`.

9. E6 — Arranque y consumo de recursos

9.1 Diseño

Se midió el ejecutable de release (`aster-desktop-assistant.exe`) en el equipo de desarrollo. El script `scripts/perf/recursos.ps1` :

1. Registra el entorno (OS, CPU, RAM, versiones).
2. Mide los tamaños del ejecutable y del frontend compilado.
3. Lanza el proceso con `Start-Process` , cronometra hasta que `MainWindowHandle` deja de ser 0 (ventana visible) y registra RAM y CPU en reposo.

Se realizó **1 corrida** (fría: primer proceso de la sesión de Windows sin haber lanzado la app antes). El parámetro `-Runs 5` está disponible para corridas adicionales en entornos de release.

9.2 Resultados

Artefactos:

Tipo	Ruta	Tamaño
Ejecutable release	`src-tauri\target\release\aster-desktop-assistant.exe`	13,54 MB
Ejecutable debug	`src-tauri\target\debug\aster-desktop-assistant.exe`	35,73 MB

Frontend compilado (`dist/`): **838,2 KB** en 4 archivos.

Archivo	Tamaño
`dist\assets\index-hsaV-MUY.js`	801,3 KB
`dist\assets\index-BoQT653X.css`	21,9 KB
`dist\assets\window-Bn9gJKIB.js`	14,3 KB
`dist\index.html`	783 B

Arranque y recursos:

Corrida	Tipo	Ventana (ms)	RAM total	RAM privada	Procesos	WebView2	CPU reposo
1	frío	**1 190,5**	398,38 MB	177,90 MB	8	7	4,98 %

Estadística	Ventana (ms)	RAM (MB)
n	1	1
Mediana	1 190,5	398,4

La RAM incluye el proceso principal y todos sus descendientes (motor WebView2).

9.3 Análisis

- **Arranque 1 190 ms** (corrida fría). Muy por debajo del umbral de 5 s y comparable con otras apps Tauri/Electron que cargan un bundle de ~800 KB.
- **RAM 398 MB** incluye los 7 procesos de WebView2 que Edge crea para sandboxing. La RAM privada del proceso principal es 177,9 MB, que es el número relevante para el consumo neto de ASTER.
- **CPU reposo 4,98 %**: roza el umbral de 5 %, probablemente por la sincronización de OneDrive activa en ese equipo y el motor de voz en espera.
- El bundle JS de 801,3 KB supera el aviso de 500 KB de Vite (ya emitido en la salida de `vite build`). No retarda el arranque perceptiblemente en este hardware, pero es el principal candidato a optimización para equipos más lentos (ver sección 10.1, prioridad 3).

9.4 Limitaciones

- La corrida fría no vacía la caché del sistema de archivos: es un arranque en frío de *proceso*, no tras reiniciar Windows.
- `MainWindowHandle` indica que la ventana existe; no garantiza que React haya pintado el primer cuadro.
- El CPU en reposo varía con la página visible, el motor de voz y la sincronización de OneDrive.

Veredicto E6: cumple. Los tres criterios (arranque ≤ 5 s, RAM ≤ 500 MB, CPU ≤ 5 %) se satisfacen en la corrida fría. El uso de RAM y CPU están en el límite superior; se recomienda repetir con `-Runs 5` en cada release para detectar regresiones.

---

10. Discusión general

10.1 Ranking de cuellos de botella

Prioridad	Cuello de botella	Evidencia	Impacto
1	Metadatos por entrada en `search_files`	E2: 4,47 s/20k; E5: p95 16,4 s	Alto: bloquea la orden más lenta
2	Fiabilidad/cuota de proveedores	E5: 33 % de éxito en `web_search`	Alto: la orden falla o espera 60 s
3	Bundle principal de 801 KiB	E6: aviso de Vite	Medio: arranque y primera pintura
4	Lanzamiento de aplicaciones	E5: p95 8,3 s	Medio: depende del sistema, no del código
5	Enrutado y orquestación	E1/E4: microsegundos	Insignificante

10.2 Lo que no hay que optimizar

- **No** hace falta mover el enrutado a Rust ni cachear `fastRoute`: ya cuesta microsegundos.
- **No** hace falta reescribir los parsers web: 76–110 µs por página es ruido frente a la red.
- **No** hace falta paralelizar el controlador: su sobrecarga es < 1 % del total.

10.3 Recomendaciones priorizadas

1. **Corregir `to_hit` / `search_files`** (E2). Comparar el nombre primero y reutilizar `DirEntry`. Volver a ejecutar `npm run bench:rust` y comparar la tabla de `rust-bench.md`; el criterio de aceptación sigue siendo 300 ms/20k.
2. **Añadir un presupuesto de tiempo a la búsqueda** y devolver resultados parciales con un aviso, en vez de bloquear hasta 20 000 entradas.
3. **Cachear búsquedas** por consulta y raíz durante la sesión.
4. **Reducir el bundle** con `manualChunks` o importaciones dinámicas para las vistas pesadas; vigilar el aviso de 500 KB de Vite.
5. **Medir E5 con IA real** usando el guion fijado, en una ventana sin 429, para separar latencia de proveedor de latencia de aplicación.
6. **Repetir E6** en cada release para detectar regresiones de arranque o RAM.

10.4 Relación entre microbenchmark y producción

E2 y E5 se validan mutuamente: el árbol sintético de 20k tarda 4,47 s y la cola real de `find_files` es de 16,4 s. La diferencia se explica porque el uso real recorre varias raíces (Documentos, Descargas, Escritorio) y puede incluir archivos de OneDrive. Que dos métodos independientes apunten al mismo componente es la evidencia más fuerte de este informe.

---

11. Amenazas a la validez

Amenaza	Efecto	Mitigación
Árboles sintéticos vs. disco real	E2 puede subestimar o sobreestimar	Se usa `%TEMP%` y se contrasta con E5
Antivirus/filtro de Windows	Aumenta el coste de abrir cada archivo	Se documenta; el orden de magnitud se mantiene
Caché del sistema de archivos	Primer acceso más lento	Criterion calienta y usa 20 muestras
Mocks en E4	No reflejan la red real	E5 mide el total real
Muestra pequeña en E5	p95 inestable	Se reporta n y se separa éxito/fallo
Datos históricos heterogéneos	Versiones y modelos distintos	Se declara como observacional
Una sola máquina	No generaliza a otros equipos	Se documenta el hardware y se deja el script
`MainWindowHandle`	No garantiza primer pintado	Se declara en E6
Benchmark JS con JIT	Puede sobreoptimizar	Se usan datos representativos y se reporta p95

12. Conclusiones

1. **El diseño local de ASTER es eficiente.** Enrutar, parsear intenciones y orquestar cuesta microsegundos; el usuario espera por la red y por el disco.
2. **La búsqueda de archivos necesita una corrección concreta.** El problema no es el recorrido BFS ni el número de raíces, sino pedir metadatos de cada entrada antes de saber si interesa. La prueba «solo carpetas» demuestra que evitar esa llamada reduce el coste por entrada 17 veces.
3. **Los parsers web no son el problema.** Procesan 500 KB en menos de 0,11 ms; la latencia y los fallos vienen de la descarga y de los proveedores.
4. **La latencia típica es aceptable; la cola no.** Conversaciones por debajo de 1,3 s y acciones nativas alrededor de 2 s conviven con una cola de 16 s en archivos.
5. **La batería queda reproducible.** Los seis comandos, los datos crudos y los scripts están versionados en `docs/perf/`; cualquier cambio futuro puede compararse contra esta línea base.

Trabajo futuro inmediato

- Implementar la mejora de `search_files` y repetir E2/E5.
- Ejecutar el guion E5 con IA real en una ventana de cuota estable.
- Añadir E6 a la checklist de release y comparar arranque/RAM entre versiones.
- Dividir el bundle principal y volver a medir el arranque.

13. Anexos

A. Archivos generados

Archivo	Contenido
`docs/perf/raw/frontend-bench.jsonl`	E1 y E4, una línea por tarea
`docs/perf/rust-bench.md` / `.json`	E2 y E3, medianas e IC de criterion
`docs/perf/resumen-metricas.{md,csv,json}`	E5, totales por acción
`docs/perf/resumen-auditoria.{md,csv}`	E5, estados y latencias
`docs/perf/raw/metrics-anon.jsonl`	E5, métricas sin texto del usuario
`docs/perf/raw/audit-anon.jsonl`	E5, auditoría sin rutas ni resúmenes
`docs/perf/recursos.md` / `.json`	E6, entorno, tamaños, arranque y RAM
`scripts/perf/report.mjs`	Genera E5
`scripts/perf/criterion.mjs`	Genera las tablas de E2/E3
`scripts/perf/recursos.ps1`	Genera E6
`scripts/perf/scenarios.json`	Guion para la corrida con IA real
`src-tauri/benches/files_search.rs`	Benchmark E2
`src-tauri/benches/websearch_parse.rs`	Benchmark E3
`src/model/ai/chat.bench.ts`	Benchmark E1
`src/controllers/AsterController.bench.ts`	Benchmark E4
`src/perf/bench-utils.ts`	Runner de tinybench y exportación JSONL

B. Privacidad

- Las claves de API nunca se leen ni se escriben en este informe.
- `metrics-anon.jsonl` elimina `utterance`; `audit-anon.jsonl` elimina `target` y `summary`. Se conservan acción, tiempos, éxito y estado.
- El informe no incluye rutas personales más allá de las que aparecen en las tablas de auditoría agregadas.

C. Declaración sobre la IA

Las pruebas E1–E4 no llaman a ningún proveedor. E5 usa datos históricos ya registrados. La corrida con IA real queda definida, pero no ejecutada por incertidumbre de presupuesto; repetirla no requiere cambios de código, solo claves válidas y el guion de `scenarios.json`.